# BookScope API Technical Report

# BookScope API Technical Report

## 1. Submission Links

- Public repository: https://github.com/IJF9GAWHY8FGA8/bookscope-api
- API documentation PDF: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/docs/api_documentation.pdf
- Presentation slides: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/slides/bookscope_presentation.pptx
- GenAI appendix: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/docs/genai_usage_appendix.pdf
- Conversation logs appendix: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/docs/conversation_logs_appendix.pdf

## 2. Introduction

BookScope API is a Django REST Framework application for book discovery, personal reading tracking, reviews, explainable recommendations, and reading analytics. The coursework goal is not only to expose CRUD endpoints, but to demonstrate database-driven API design, data ingestion, authentication, testing, documentation, and version-controlled delivery.

## 3. Problem and Motivation

Many public book APIs provide metadata, but they do not directly support user-centered reading workflows such as maintaining a local bookshelf, capturing private reading states, writing platform-specific reviews, or generating recommendations from a user's own behavior. BookScope addresses that gap by combining third-party metadata with locally stored user activity. This design also aligns well with the assignment brief because it supports both operational CRUD use cases and higher-level analytical endpoints.

## 4. Technology Choices

### Django and Django REST Framework

Django was selected because it provides a mature ORM, authentication foundation, admin interface, migrations, and a structure that supports clear separation between models, serializers, views, and services. Django REST Framework extends this with authentication classes, serializers, pagination, filtering, and schema generation, which makes it suitable for a coursework project that needs both fast implementation and defendable architecture.

### SQLite

SQLite was selected as the database because it is lightweight, zero-configuration, and fast to work with during a short coursework cycle. It is appropriate for a single-user or small-scale API demonstration and simplifies local setup for markers. Its main limitation is that it is not the best choice for highly concurrent production systems, which is acknowledged as a tradeoff rather than ignored.

### Google Books as External Data Source

Google Books was used as a metadata source rather than as the direct runtime backend. This distinction is important. The system imports and normalizes selected Google Books metadata into local tables, allowing the API to remain database-driven and to support user activity that third-party services do not model for this coursework. This approach also improves reproducibility because the core API behavior depends on local data, not on live external API responses.

## 5. System Architecture

The system is split into four functional areas:

- `accounts` for registration, JWT authentication, and current user access
- `catalog` for books, authors, genres, search, filtering, and ingestion
- `engagement` for bookshelf entries and reviews
- `analytics` for recommendation and aggregate endpoints

This modular split improves maintainability and makes it easier to explain the system during the oral examination.

## 6. Data Model

The main entities are:

- `Author`
- `Genre`
- `Book`
- `BookshelfEntry`
- `Review`

The catalog uses many-to-many relations between books and both authors and genres. Bookshelf entries and reviews connect a user to a book. Two explicit uniqueness rules are enforced:

- one bookshelf entry per user per book
- one review per user per book

These constraints were added to preserve consistency and to simplify downstream recommendation and analytics logic.

## 7. Data Ingestion and Normalization

The ingestion pipeline fetches Google Books payloads, extracts useful fields, normalizes identifiers and text values, and stores the result locally. The process includes:

- extracting title, subtitle, authors, categories, publisher, language, page count, and links
- mapping ISBN 10 and ISBN 13 when available
- parsing publication year from raw publication dates
- creating or updating local authors and genres
- linking imported books to their related entities

This data preparation stage is important because public datasets are rarely perfectly clean. Handling missing identifiers, multiple authors, multiple categories, and inconsistent date formats demonstrates practical engineering rather than idealized CRUD scaffolding.

## 8. API Design

The API follows a REST style with JSON responses, pagination, filtering, and clear permissions. Public endpoints expose catalog browsing and selected analytics. Authenticated endpoints enable user-specific actions such as bookshelf management, review creation, personalized recommendations, and reading summaries. Administrative write access is restricted to staff users for catalog management.

Key endpoint groups include:

- authentication
- catalog CRUD

- bookshelf CRUD
- review CRUD
- recommendation
- analytics

This structure supports both the minimum coursework requirements and higher-scoring analytical behaviors.

## 9. Authentication, Validation, and Error Handling

JWT was chosen because it works well for stateless API authentication and is easy to demonstrate in local and hosted environments. Validation is handled through serializers and model constraints. The API also wraps errors in a structured response format to make failures easier to interpret and document. Typical validation examples include rating limits, duplicate bookshelf entries, duplicate reviews, and permission checks.

## 10. Recommendation and Analytics Design

The recommendation engine is intentionally rule-based rather than machine-learning-heavy. This was a deliberate tradeoff. A rules approach is faster to implement, easier to test, and much easier to justify in a short oral exam because recommendation reasons can be surfaced directly. The scoring strategy combines:

- preferred genres inferred from highly rated books
- preferred authors inferred from highly rated books
- external rating strength
- external rating count
- local popularity

The analytics endpoints complement this by exposing catalog-level and user-level summaries such as genre popularity, ratings distribution, top authors, and reading summary metrics.

## 11. Testing Strategy

Testing focused on the highest-risk behaviors:

- registration and authenticated access
- catalog list and create workflows
- bookshelf uniqueness rules
- review uniqueness rules
- recommendation exclusion logic
- analytics response correctness

This approach was selected because the coursework emphasizes working code, error handling, and defendable implementation choices rather than raw test volume alone.

## 12. Challenges, Lessons, and Version Control

The main implementation challenge was balancing coursework scope against depth. A fully ML-driven recommender or a large production database would have expanded complexity without clearly improving the coursework outcome. The chosen design instead prioritizes a clear domain model, explainable recommendation logic, consistent validation rules, and a staged commit history. Version control was intentionally organized around runtime setup, data ingestion, tests, source documentation, and generated assets so that development progress remains inspectable in the public repository.

## 13. Deployment and Delivery

The repository includes a PythonAnywhere deployment package built around a manual WSGI web app, a virtualenv, static file mappings, and an environment-driven configuration file. This choice fits the coursework well because it provides a simple externally hosted Django deployment without requiring Docker or more complex infrastructure. In this environment no PythonAnywhere account credentials were available, so a live public URL could not be provisioned automatically. The deployment guide and WSGI template are nevertheless included so that the project can be published on PythonAnywhere with minimal manual setup.

## 14. Limitations

The project has several acknowledged limitations:

- SQLite is not ideal for high-concurrency production workloads
- recommendation scoring is heuristic rather than learned from large-scale behavior
- trend scoring is simplistic and can be improved with richer temporal weighting
- live hosting still depends on final PythonAnywhere account-side setup

Stating these limitations explicitly is important because the brief expects reflection and awareness of future improvement areas.

## 15. Future Improvements

Potential future work includes:

- stronger ranking signals and configurable recommendation weighting
- richer search and faceting
- caching for analytics endpoints
- deployment hardening and environment-specific settings
- more advanced documentation export workflows

## 16. GenAI Declaration and Analysis

Generative AI was used as a declared planning, implementation, review, and documentation support tool. It was used to extract the coursework requirements from the brief, compare architecture options, shape the Google Books ingestion strategy, propose tests, and accelerate submission material drafting. Its outputs were not accepted blindly. Weak suggestions were rejected, several high-level plans were narrowed to fit a two-day build window, and the final module split, recommendation strategy, and deliverable structure were selected manually. The detailed declaration is in the GenAI appendix and the representative interaction record is provided in the conversation logs appendix.

## 17. References


- Google Books API documentation: https://developers.google.com/books/docs/v1/using
- Django documentation: https://docs.djangoproject.com/
- Django REST Framework documentation: https://www.django-rest-framework.org/
- Simple JWT documentation: https://django-rest-framework-simplejwt.readthedocs.io/
- drf-spectacular documentation: https://drf-spectacular.readthedocs.io/